

Element 1 – UML Modelling & Design Patterns

-

Module Title: Applied Software Engineering

Module Code: CP60019E

Assignment: Assignment 1 – UML Design

-

Student Name: Mohammed Adnan Osman

Student ID: 33114153

Course: BSc (Hons) Computer Science

Academic Year: 2025 – 2026

-

Module Leader: DR Nasser Matoorianpour

Table of Contents

Task 1	2
Use case model.....	2
Task 2.....	7
Class Diagram.....	7
Task 3.....	10
Sequence Diagram	10
Task 4.....	13
Design Patterns	13
Task 5.....	15
Component Models	15
Task 6.....	18
Project Risk Management.....	18

Task 1

Use case model.

Develop a Use case model for the information provided for ONE use case. As part of your answer produce the use case description and use case scenario steps. Include appropriate use case refactoring. [10 marks]

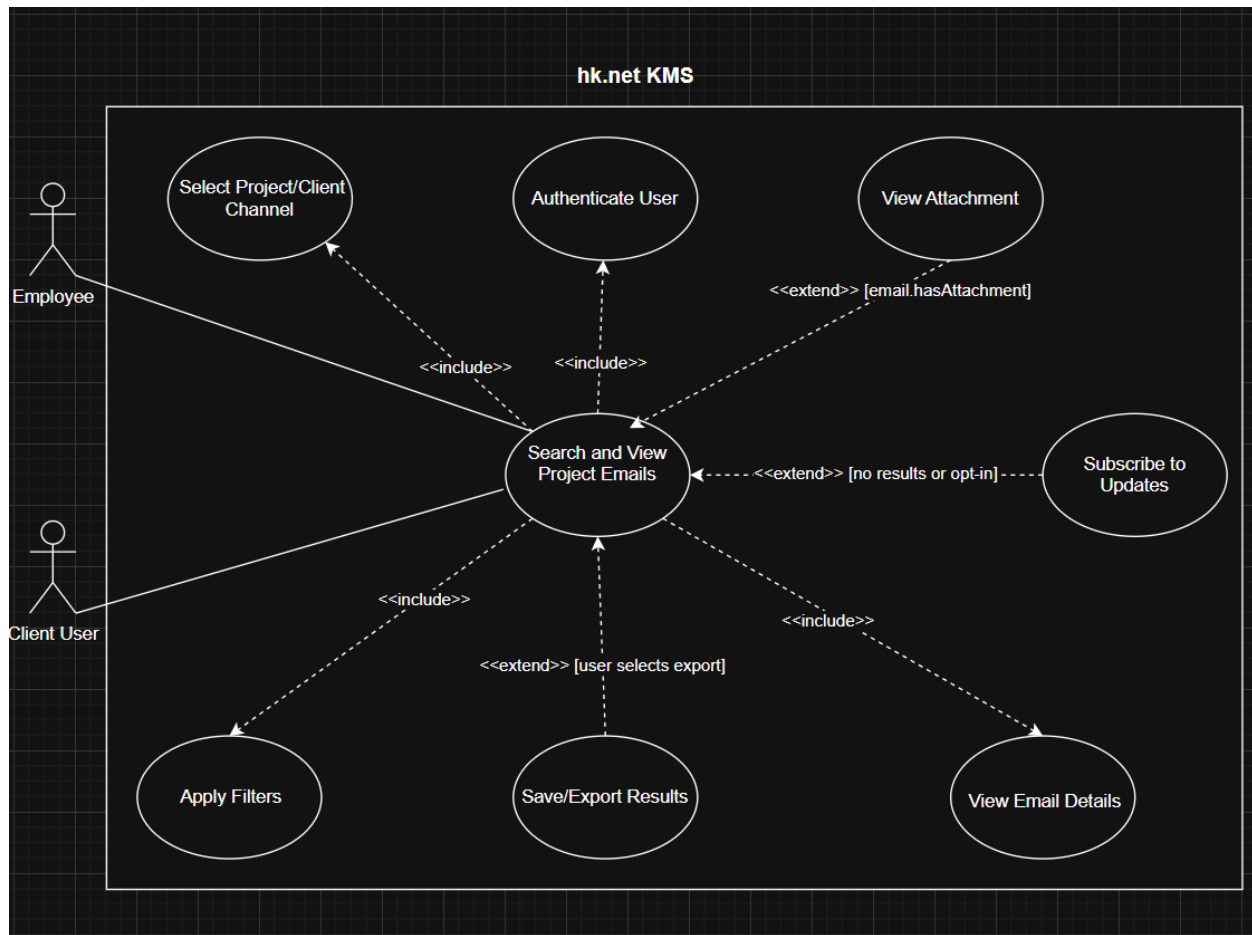


Figure 1: Use Case Diagram for hk.net KMS – Draw.io

The use case diagram focuses on the scenario “Search and View Project Emails”. Employee and Client User interact with hk.net to get emails and open attachments. Mandatory steps are factored with <<include>> and optional avenues uses <<extend>> with guards, keeping the model modular and clear. A use case models shows how a user interacts with the system.

Use Case Model: Search and View Project Email

System: hk.net Knowledge Management System (KMS)

Primary Actor: Employee (H&K Staff)

Supporting Actors:

1. Client User – Read only access to authorized project channels.
2. Authentication Service – External system/ Service
3. Email/ Document Indexer – Background service.

Scope: Searching and viewing archived project emails and attachment indexed by project/ team/ client.

Goal: Quickly find and view archived project emails and allow attachments scoped to a project or client channel to enable fast onboarding and reuse knowledge.

Use Cases:

Use Case: Search and View Project Emails

Use case sub cases:

1. Authenticate User (included)
2. Select Project/ Client Channel (included)
3. Apply Filters (included)
 - Keywords
 - Dates
 - Sender
 - Tag
 - Attachment Type
4. View Email Details (included)
 - Show metadata.
 - Content preview
5. View Attachment (extend)
 - Optional if email has attachments.
6. Save/ Export results (extends)
 - Optional (CSV/ PDF export or save search query)
7. Subscribe to Updates (extends)
 - Optional – registers for notifications when new matching emails arrive.

Use Case Description

Use case: Search and View Project Emails

Primary Actor: Employee

Stakeholders and Interests:

1. Employee: Fast, accurate retrieval of past communications
2. Client User: Read only access to their channel's materials.
3. Management: Better adoption, faster onboarding and preserving original memory.
4. Security/Compliance: Properly authorized access and auditability.

Pre Conditions:

1. A valid hk.net account exists.
2. Credentials are active.
3. Project emails are indexed and linked to the correct channels.

Trigger:

1. The user needs background on a project/ topic.
2. Initiate a search.

Main Success Scenario (Basic Flow):

1. User opens hk.net.
2. System requests login and user authenticate. (include: Authenticate Users)
3. User selects a Project/Client Channel. (include: Select Project/Client Channel)
4. System displays search UI with keywords and filters.
5. Users enter keywords and apply filters:
 - Date range
 - Sender
 - Tags
 - Has attachment
6. System queries the index and returns a paginated list.
 - Subject
 - Snippet
 - Date
 - Sender
 - Attachment indicator
7. User selects a result. (include: View Emails Details)
8. System shows full email content/metadata and any permitted attachments.
9. Optional: User opens/ downloads an attachment. (extend: View Attachment {email, hasAttachment})
10. Use case ends.

Alternative/Exception Flows:

1. Invalid credentials [Step 2]:
 - > Show error
 - > Offer password reset
 - > Retry Login
2. No matching results [Step 6]:
 - Broaden keywords.
 - Remove filters.

- Offer Subscribe to updates when new matching emails arrive (extend: [no results or opt-in])
- 3. Insufficient permission [Step 3]:
 - > Deny access
 - > Offer request-access workflow
- 4. Low bandwidth/ timeout [Step 6]:
 - Return lightweight snippets/ smaller page size.
 - Users can retry/ refine.
- 5. Save/ Export [After step 6]:
 - User saves query or exports results (extend: [user selects export])

Postconditions (Success Guarantee):

1. User views the content/ preview at least one matching email.
 - Optional - User downloads attachments accessed.
 - Optional - User saves search/ subscribes to updates.
2. Actions are audit- logged.

Minimal Guarantees:

1. No unauthorized data is disclosed.
2. Any failed attempts are logged in audit.

Special Requirements/ Constraints:

1. Indexed search should return to typical queries.
2. Pagination or infinite scroll for large result sets must be supported.
3. Audit logging on search and view.
4. Accessible UI and results
5. Varying site connectivity - design degrades nicely.
 - Smaller Pages
 - Previews

Refactoring Rationale:

1. <<include>> (Mandatory/ Reuseable) – They are always part of the use case.
 - Authenticate User - Common cross cutting login requirement reused by many use cases.
 - Select Project/ Client Channel - Ensures correct scoping and authorization before search.

- Apply Filters/ View Email Details – Mandatory steps within the main flow factored for reuse in other related use cases such as search documents or other search style user case.
- 2. <<extend>> (Optional/ Conditional) – These happen only in certain conditions or by the user's choice thus are kept outside of the main flow and guard them. Helps in reusability, which is a key component of object-oriented programming style.
 - View Attachment
 - Save/ Export results.
 - Subscribe to updates.

Glossary

Channel: A client or a project specific workspace in hk.net controlling access and the scope.

Indexer: A service that extracts and stores searchable metadata and content for attachments and emails.

Subscriptions: Saved queries and channels alert that notifies users when matching new content arrives.

Task 2

Class Diagram

Produce a class diagram with appropriate principles of object orientation applied, related to the use case model in Task 1. As part of your model, produce a system class with clear set of public methods. [20 marks]

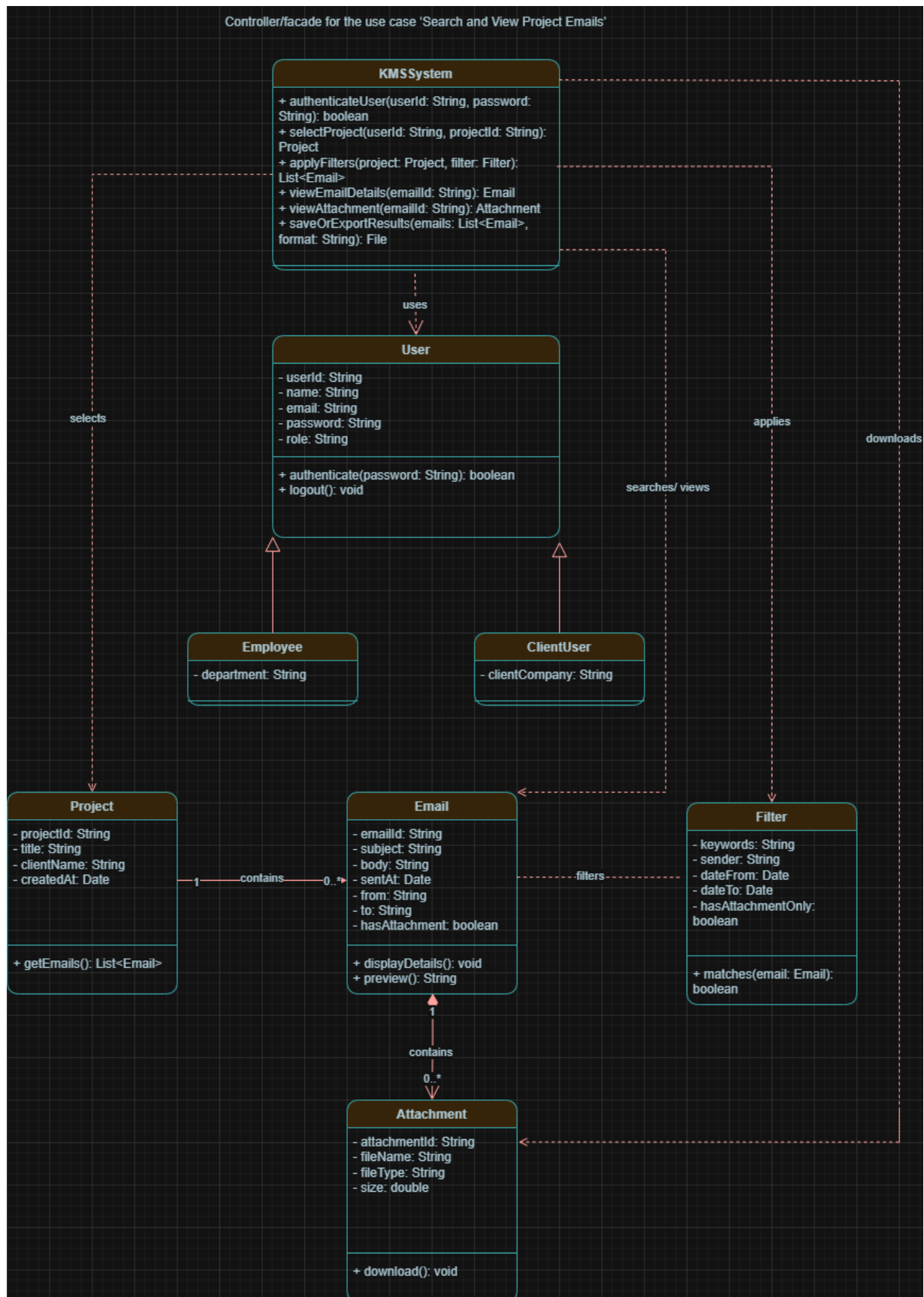


Figure 2: UML Class Diagram for the “Search and View Project Emails” Use Case in hk.net KMS – Draw.io

Have you provided a class diagram which captures all elements of the system?

The KMSSystem class acts as the central point of entry which provides all the necessary functions like searchEmail or viewAttachment. All the objects are present with their own specialized subclasses like Project and Email. The filter class encapsulate the search criteria which is important for the applyFilters use case step.

Do you explore the issue involved in the engineering task required and software design required?

For access control, it solves it using inheritance but specializes user into employee and ClientUser and implementing KMSSystem methods to ensure role-based access before proceeding. To ensure attachments and emails can't exist without their container, I used composition relationships like Project to Email and Email to Attachment. This ensures data integrity and is critical when implementing this type of archival system. To ensure maintainability, I've used a modular approach using a controller pattern which helps to separate the concerns. This KMSSystem separates the data storage mechanism as shown with the abstract interfaces in the diagram to make the system easier to understand and to build upon it.

Have you understood the main functions that must be provided by the system?

The public methods from the system align with the use cases I've done:

1. Authenticate (include): Handled by +authenticateUser()
2. Select Project/ Client Handles Chnnel (include): Handled by +selectProject()
3. Apply Filters (include): Handled by +applyFilters()
4. View Email Details (include): Handled by +viewEmailDetails()
5. View Attachment (Extension): Handled by +viewAttachment() and +download() on the Attachment class.
6. Save/ Export (Extension): Handled by +saveOrExportResults()

Have you documented your engineering analysis findings?

The class diagram above represents the object-oriented design of the hk.net system for the “Search and View Project Emails” use case. It shows how the different interactions work for the use case I picked. The diagram includes core classes such as User, Employee, ClientUser, Project, Email, Attachment, and Filter, as well as the main controller class KMSSystem. These classes collectively describe how employees and clients can authenticate, access project data, and retrieve archived emails and attachments efficiently.

The KMSSystem class functions as the controller within the architecture, coordinating all the operations in the system. It provides public methods that enable authentication, project selection and email searching through filters, viewing detailed email content, accessing attachments, and exporting search results. The relationships between classes show how the system components interact: Project contains multiple Email objects, each Email can have an optional Attachment, and Filter defines to outline the specific search result needed. This keeps the model modular and easy to extend for future add Ons if needed.

Inheritance is shown through the user class which has shared attributes compared to employee or clientuser extend it for specialized behavior. Composition is applied between Email and Attachment, showing that attachments depend on the existence of their parent email. Encapsulation is used by defining private attributes and public operations, ensuring data security and abstraction. The design also shows separation of concerns, where the system manages system logic while domain classes focus on representing business entities.

Have you applied the required level of abstraction in your models?

For generalization the User class is abstract as it contains only shared attributes, which abstracts the common behavior before it separates into Employee and ClientUser. By making the attribute private and making the operation public, the internal complexity is abstracted away as the system doesn't need to know how User authenticate only that it can all the +authenticate(password). This is encapsulation. I used to state the data types for implementing details.

Task 3

Sequence Diagram

Produce a Sequence Diagram for the selected use case. Include possible guards, iteration, and message operations in your diagram. [10 marks]

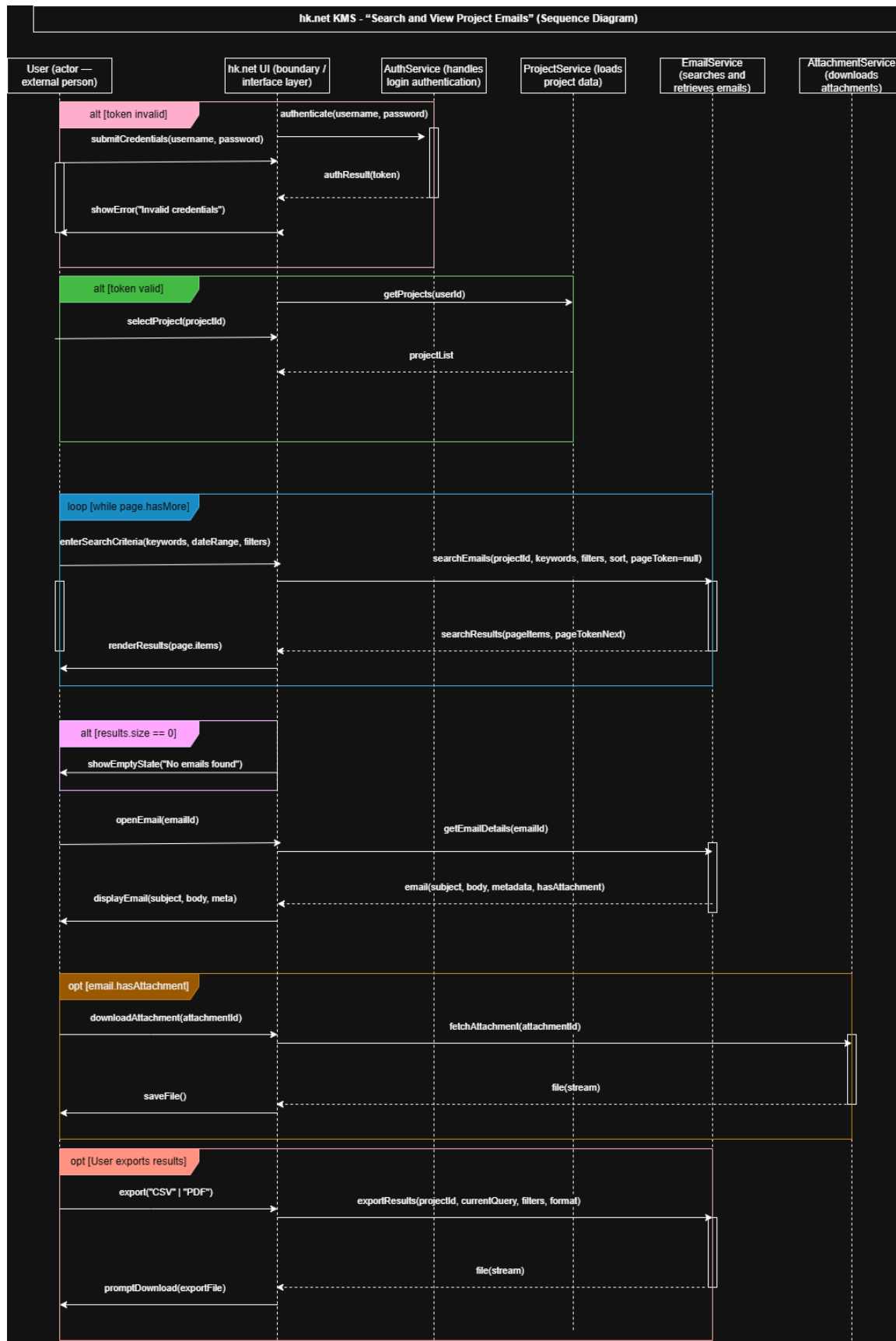


Figure 3: Sequence Diagram for hk.net KMS – “Search and View Project Emails” – Draw.io

Have you produced a Sequence Diagram for the selected use case?

Yes, I showed objects and lifelines of: User, UI, AuthService, ProjectService, EmailService, AttachmentService and the sequence of the messages exchanged between them over time. It shows the runtime interactions between the user, user interface and the services to get the project emails. It starts with an authentication, followed by a project selection using alt [token valid/ invalid] branch. After this, the user enters filters and keywords for the search phase. Then the UI goes to the EmailService which is inside the loop [while page.hasMore] which pages the results. Conditional guards are used to change the behavior like alt[result.size == 0] which shows that no results were found message. The normal flow will continue to openEmail(emailId) to get the details of the message. There are also optional fragments like opt[email.hasAttachments] for downloading attachments or opt[user exports results] for exporting the search data.

Have you included possible guards, iterations and message operations in your diagram?

I've used guards, which is a Boolean expression that controls the process flow using the alt fragment with guard conditions to make the conditional logic such as: [taken invalid] / [taken valid], [result size == 0], [email has Attachment] and [User exports results].

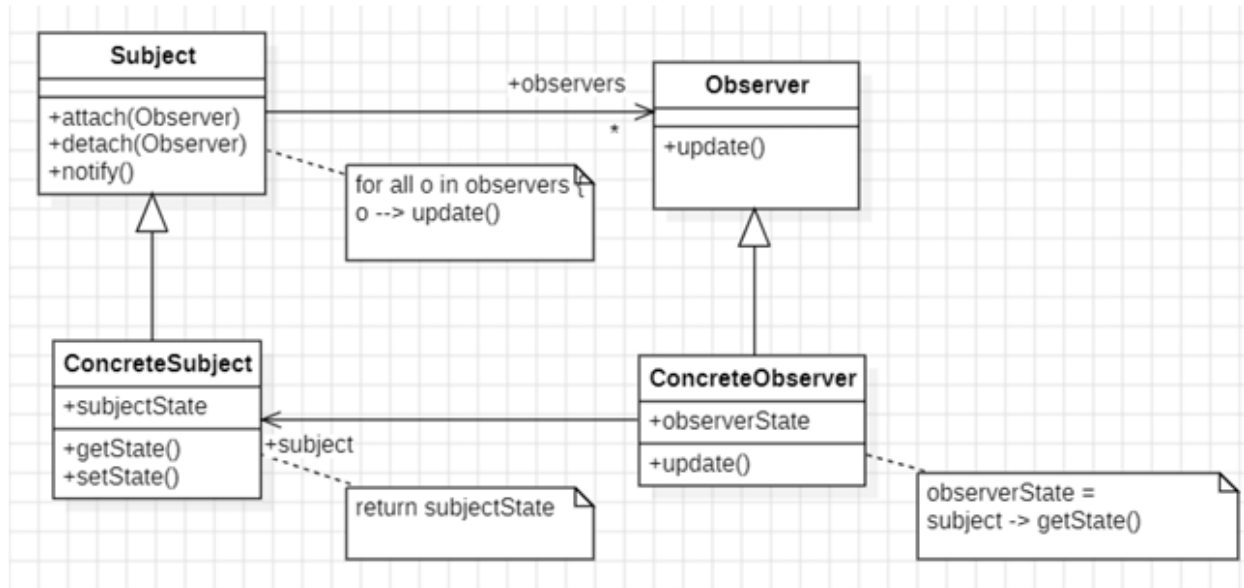
- [taken invalid] / [taken valid] – (Alternative) This is the authentication outcome. If security token is valid then it allows access to the projects, if not will show an error.
- [result size == 0] – (Alternative) This is the outcome of the search, if search found no results, then message of no emails found and will show the main path if results are found.
- [email has Attachment] – (Optional) This is the conditional action; this will only be executed if specific email which the user is viewing has an attachment so they can download to save it.
- [User exports results] – (Optional) This is the optional user action where they can export pdf or csv if the user chooses it after viewing the search results.

For iterations I have used a loop fragment to allow for repetition. For example, loop [while page.hasMore] for paginating search results.

- Loop [while page.hasMore] – (Loop) The messages inside this loop of searchResults and renderResults are repeated if the guard condition, the loop, is true which is needed when handling the many pages of search results.

Task 4

Design Patterns



Observer pattern defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically: Ref: Gamma et al. Apply the observer pattern (by modifying the classes in the above structure) to your chosen use case and class diagram which you have developed. [10 marks]

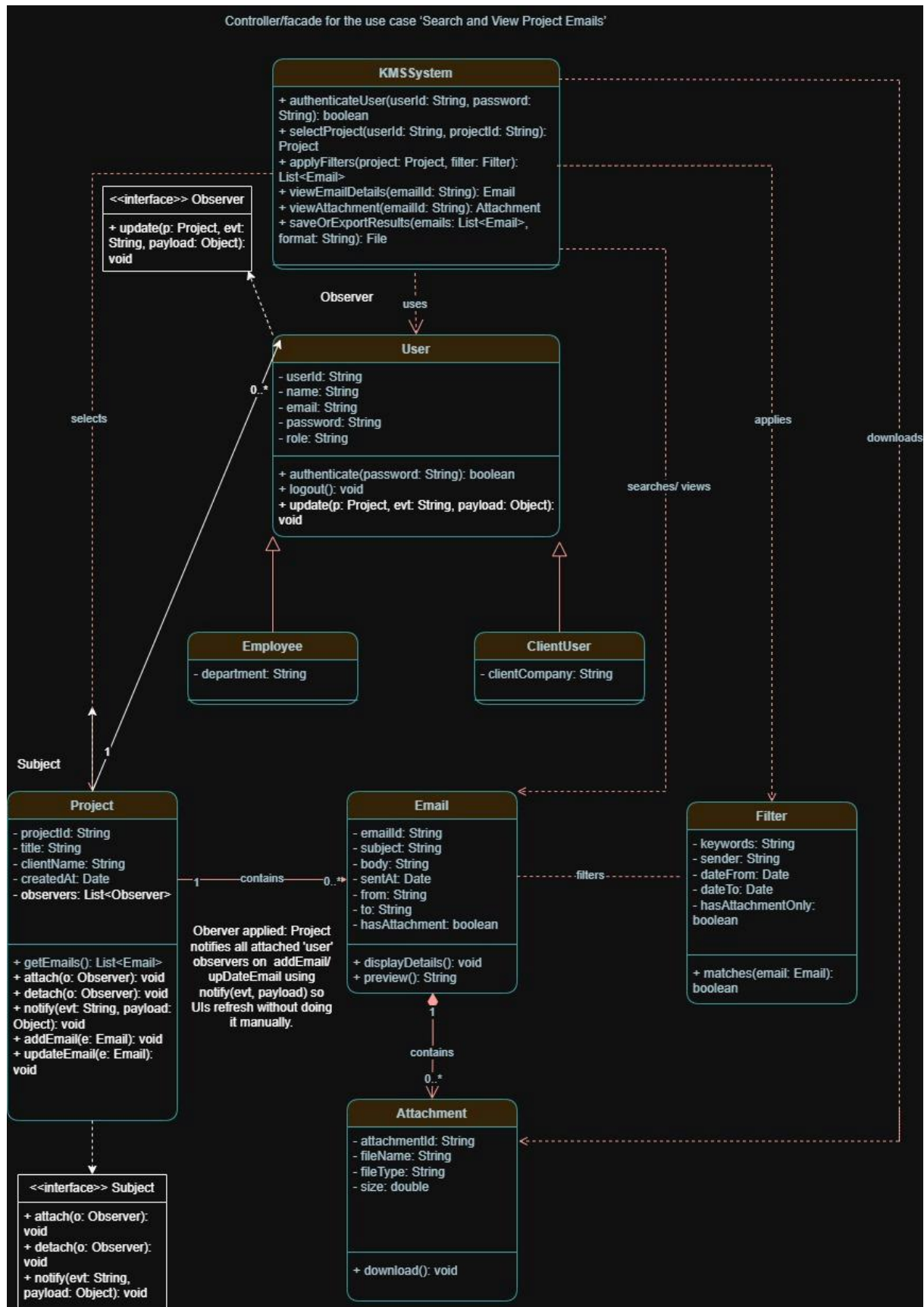


Figure 4: Application of the Observer pattern to the hk.net KMS system. – Draw.io

Have you produced a model to represent the specified Design Patterns?

The diagram I made implements the observer pattern by defining the project class as the Concrete Subject, maintaining a list of user class as the Concrete Observer. This allows the user to subscribe to specific projects and to receive automated notifications when there are changes to it. This ensures the user interface remains synchronized with the project data. The user and its subclasses of employees and client users define the update method. So, whenever the new email or attachment is added to the project, notify (evt, payload) calls the update () on all the observers. This makes every user or client display the latest project announcement information without manual refreshing.

By having a centralized system of KMSSystem, it uses a façade pattern which acts as a central point of entry for all the use cases. This simplifies the user and client interaction by separating the user interface and back-end services. This demonstrates the use of abstraction and making it more reuseable in a component software design.

Task 5

Component Models

Produce ONE Component with associated interface methods to represent the class diagram model. As part of your solution, illustrate various possible component compositions and discuss the significance of your design. [20 marks]

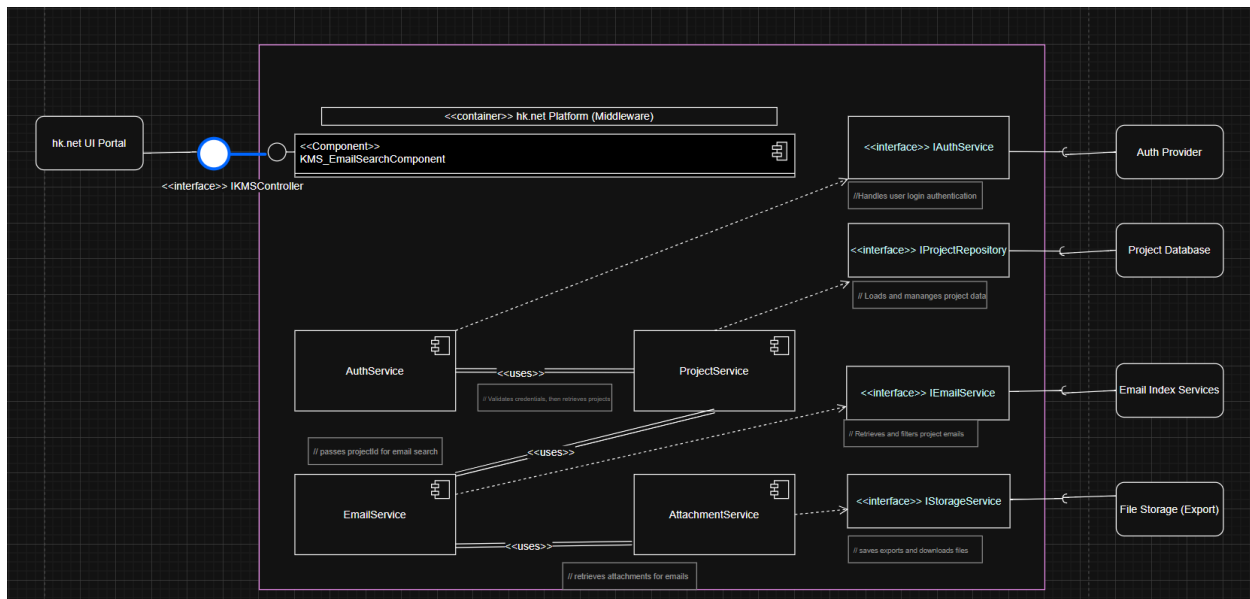


Figure 5: Internal Structure of KMS_EmailSearchComponent showing provided and required interfaces, internal subcomponents, and connections to external services – Draw.io.

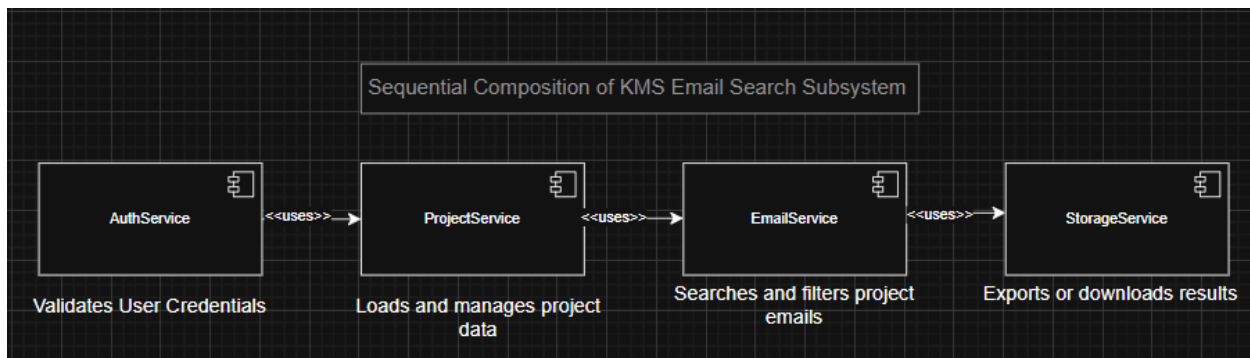


Figure 6: Sequential Composition Variant showing the linear flow – Draw.io.

Have you produced a suitable Component and associated interface and component composition?

The overall system is named KMS_EmailSearchComponent, which acts as the facade by encapsulating the internal services such as AuthService, ProjectService, EmailService etc within the middleware. It has a central entry point for the client as hk.net UI Portal.

The system is accessed externally using the <<interface>>IKMSController interface which is the public contract for the entire search component. Internally, the comonenets depend on the required interfaces of IAuthService, IProjectRepository, IEmailService and IStorageService.

The top diagram shows the aggregation which comprises several specialized services, and figure 5 shows the sequential composition which shows the linear flow.

Have you related your component to the classes in your class diagram?

The system component directly mirrors the class diagram I made. The component boundary and the system both act as the controller. They serve as the single public entry point coordinating all the operations defined in the interface. The public method of the system class such as +applyFilters() or +viewAttachments represent the aggregated functionality shown to the outside world.

Component	Related classes	Functionality
AuthService	User Employee ClientUser	This logic is for validating the credentials for the user by executing the <code>KMSSystem.authenticateUser()</code> method.
ProjectService	Project	This is for loading, managing and restricting access to project data using methods like <code>KMSSystem.selectProject()</code> .
EmailService	Email Filter	This logic is for searching and retrieving data from the interaction between the <code>KMSSystem.applyFilters()</code> method and Filter class.
AttachmentService	Attachment	This logic is for file operations like downloading or exporting it using methods: <code>KMSSystem.viewAttachment()</code> and <code>saveorExportResults()</code> .

Have you applied appropriate component compositions?

For the sequential composition type I've shown the rigid sequence to demonstrate the flow to satisfy the use case goal. It ensures preconditions are met at each step of the way: ProjectServices requires validation from AuthService. EmailServices require the project approval using the ProjectService and StorageService requires the search result to match EmailService.

The aggregation shows the component containing and relying on smaller specialized components which promotes separating functionalities into distinct components. This helps in achieving modularity where each component is tested and developed independently of each other, thus easier to maintain. As each depend on an interface, it allows for one implementation to occur without affecting the service.

Task 6

Project Risk Management

In relation to the use case in Task 1, identify at least one risk which may occur during the completion of the system by using the Risk-Register template and how to control the identified risk. [10 marks]

Risk No.	Risk Description	Category	Likelihood (1-5)	Impact (1-5)	Risk Level	Mitigation/Control Strategy	Responsible Person	Status
1	Inaccurate or outdated email data may reduce trust and usability of the System	Data Quality	3	5	High	Apply validation and indexing, add external validation, have data audits.	Data Administrator	Open
2	Low user adoption due to low training or incentives.	Personal shortfall / Organizational	4	5	High	Deliver training, add incentive programs, integrate usage in performance reviews.	Project Manager	Open
3	Bad system performance or loading speed in low-bandwidth offices.	Technical Risk / Real - time lackluster performance	3	4	Medium	Implement regional hosting, coaching, and lightweight result views.	Technical managers	Open
4	Unauthorized access or potential data breach exposing client private information.	Security / External task shortfall	2	5	High	Implement encryption, access controls, Robust authentication systems, and regular security audits.	Security Officer	Open

Identify at least one risk which may occur during the completion of the system by using the Risk-Register template and how to control the identified risk.

To explicitly discuss one, I'll mention risk number 1. Here the strategy would be to avoid the risk of inaccurate or outdated email data by applying data governance and robust validation.

This falls under the category of data quality which is extremely high importance and also exceedingly elevated risk. Due to data being primarily done by humans, manual errors are almost inevitable and neglecting updating the information. When transferring data from one system to another, there is a chance that data translation or formats of them could change resulting in bad data. This collectively gives a score of 3 out of 5 in terms of likelihood. This would have extremely high impact as hindrance to the data will lead to a loss of trust and usability and the system's operational status would be considered a failure as it isn't serving its purpose. The idea of encouraging the users to interact with the system by using it and making it more efficient will damage the reputation and using it leads to critical errors and may breach confidential information. This justifies giving it a score of 5 out of 5.

One way to combat this risk would be to make a required format validation to ensure all emails have the required syntax such as containing the '@' symbol or requiring domain extensions such as '.com'. These will be mandatory for all the users interacting with the system, which should prevent missing information. If these formats end in the same email, it will flag and reject duplicated email which prevents data inconsistency. May also add external verification, if possible, to confirm email domains and accept the emails which will also reduce dead email addresses, which aren't currently used. Data audits to check for human error during manual login entry and to ensure consistency by having a standard format.

This is to be handled by the data administrator, or the employed data audit team as they possess the access and authority over this technical knowledge and hold an important position when it comes to implementing validation, conducting audits and enforcing the strict data governance to negate the risk effectively. The status of this is open as this risk is always a possibility and needs to be monitored continuously.